

中文精读批注版

HScope 中文译注与逐步精读

用途：零基础逐步精读、课堂汇报与方法复现准备。

说明：非官方学习译注；保留原文章节、图表号与关键数字，正式引用以英文原文为准。

制作日期：2026-08-26 · Asia/Singapore

版本说明 本文依据作者公开的 22 页 ISSTA 2026 论文制作，是按原文章节和图表组织的非官方学习译注。为便于零基础理解，采用忠实译读与解释性压缩，不替代 HScope-英文原文-ISSTA-2026.pdf。

论文信息

- ☐ 英文题目: Fine-Grained Privacy Leakage Detection in OpenHarmony Apps
- ☐ 中文学习题目: OpenHarmony 应用中的细粒度隐私泄漏检测
- ☐ 作者: Aohan Mei、Guangliang Yang、Xinming Guo、Yi Wang、Fuan Gui、Min Yang
- ☐ 出处: ISSTA 2026
- ☐ 一句话: 直接分析无类型 Ark 字节码，通过抽象解释、框架建模和污点传播恢复敏感数据从 Source 到 Sink 的可能路径。

贯穿案例

组件 C1 点击后读取 OAID 并写入状态 `message`；C1 创建组件 C2，把同一个状态和回调传给它；C2 点击按钮时通过回调把 `message` 传回 C1 的 `request`，最终进入 `http.post`。源和汇不在同一函数，也不在同一组件。

<pre> 1 @Entry @Component 2 struct C1 { 3 @State message:string = 'no taint' 4 request(message){ 5 http.post(message) //Sink API 6 } 7 build() { 8 RelativeContainer() { 9 //Initialize component2 10 C2({callback:(msg)=>{this.request(msg)}, 11 message:this.message}) 12 Text(this.message).onClick() => { 13 //Read source data 14 this.message = identifier.getOAID() 15 }) 16 } </pre>	<pre> 17 @Component 18 struct C2 { 19 callback = (): void => { } 20 @Link message:string 21 22 build() { 23 RelativeContainer() { 24 Button().onClick()=>{ 25 //Call sink API 26 this.callback(this.message)) 27 28 } </pre>
--	--

Fig. 1. Simplified Real World Privacy Issue (Case#1)

components C1 and C2. A naive analysis would fail to connect the source to the sink, as the data

HScope-图1-跨组件泄漏案例

图1看图顺序 先找第14行 `getOAID()`，再看第10—11行如何把回调和共享状态交给 C2，最后看第26行回调怎样回到第5行 `http.post`。难点不是看见 Source/Sink，而是连接中间别名与间接调用。

第一单元：摘要与引言——为什么 Android 工具不能直接搬来

原文摘要译读

OpenHarmony 快速普及后，隐私滥用与数据泄漏成为重要问题。Ark 字节码缺少静态类型且行为灵活，使现有分析方法难以精确解析间接调用和复杂组件通信。论文提出 HScope：直接分析 OpenHarmony App 字节码，通过抽象解释模拟程序行为，恢复间接调用与跨组件通信，并执行细粒度隐私数据流检测。

作者在 300 个真实 OpenHarmony App 上评估系统，报告 39 个此前未知、经人工确认的隐私问题，涉及 27 个 App。结果表明，HScope 能为不断扩张的 OpenHarmony 生态提供可扩展的静态隐私检测基础。

引言译读

OpenHarmony 使用 ArkTS/JavaScript 风格开发并编译为 Ark 字节码。与 Android 的 Java/Dalvik 生态不同，字节码层可能丢失类型，函数可作为值传递，闭包、回调和共享状态十分普遍。因此，先建立固定调用图再做污点传播容易漏掉真实调用；只在单函数内找 Source 和 Sink 更会漏掉跨组件路径。

三个新术语

- ☐ Ark Bytecode (Ark 字节码)：OpenHarmony App 编译后的低层指令。
- ☐ Indirect Call (间接调用)：被调用函数由变量或回调决定，代码中没有固定目标名称。
- ☐ Inter-component Communication (ICC, 组件间通信)：页面或组件通过回调、路由或共享状态传递数据和控制。

批注：论文的核心困难

HScope 不是“再写一个 Source/Sink 搜索器”。真正贡献是让无类型、回调密集、组件化的程序重新拥有足够的语义关系，然后污点分析才有机会看见完整路径。

最少记忆清单

- ☐ OpenHarmony 的难点来自无类型字节码、间接调用和 ICC。
- ☐ 先恢复程序语义，再做污点分析。
- ☐ 静态报告的是可能路径，不是某次运行已经泄漏。

主动回忆题

1. 为什么只看到 `getOAID` 和 `http.post` 还不够？
2. 回调会怎样破坏预先固定的调用图？
3. HScope 与动态抓包的结论边界有什么不同？

进入下一单元检查点

- ☐ [] 能用图1画出五节点数据流。
- ☐ [] 能解释无类型字节码为什么增加分析难度。
- ☐ [] 不把静态可达说成真实发生。

第二单元：第2节——OpenHarmony 编程模型与真实案例

2.1 编程模型译读

OpenHarmony 应用由 Ability 和 UI 组件组成。ArkUI 采用声明式界面；事件回调、装饰器状态和路由跳转决定运行时控制流。状态变量可能在父子组件之间通过链接共享，函数也可以作为回调传递。对静态分析而言，数据关系常藏在框架约定中，而不是显式普通赋值。

2.2 真实案例译读

论文用两个组件说明传统分析为什么失败。C1 的 `message` 最初无污点；用户点击文本后，OAID 被写入该字段。C1 构造 C2 时把 `message` 与匿名回调一起传入；C2 的按钮点击处理器调用回调。回调闭包最终调用 C1 的 `request`，把共享字段送入 `http.post`。

若分析器不知道 `@State` 与 `@Link` 指向同一抽象对象，污点会在组件边界消失；若不知道 `this.callback` 的实际目标是父组件传入的匿名函数，调用图也会断开。

三个新术语

- ☐ State Alias（状态别名）：两个字段名在运行时可能指向同一个状态对象。
- ☐ Callback（回调）：把函数作为值交给另一组件，之后由对方触发。
- ☐ Closure（闭包）：函数携带创建位置的变量环境，使回调能够访问父作用域对象。

最少记忆清单

- ☐ 共享状态决定数据边，回调决定控制边。
- ☐ 两类边缺一条，Source-to-Sink 路径都会断。
- ☐ 框架语义必须显式建模，不能只靠通用语法。

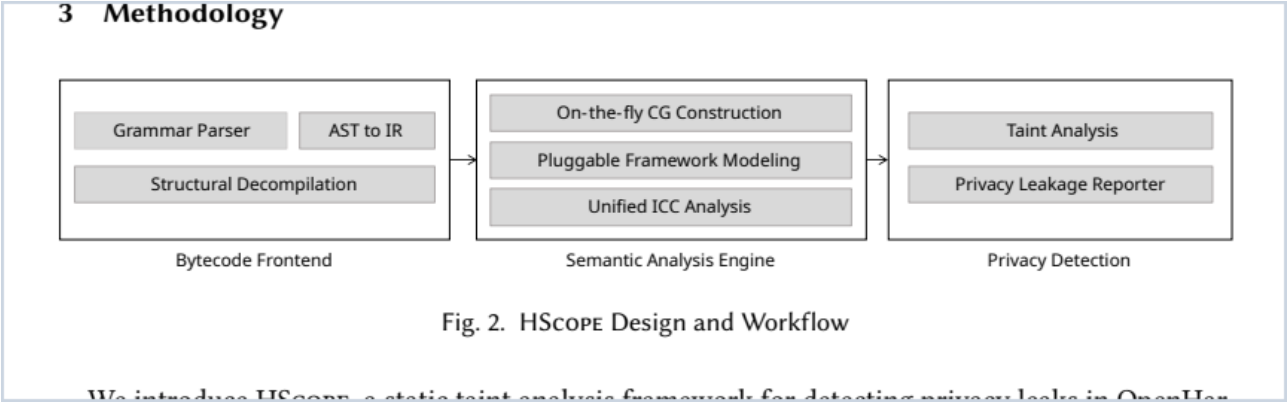
主动回忆题

1. `@Link message` 为什么不是普通复制？
2. 回调目标不明会影响调用图还是污点标签，还是两者都影响？
3. 如果 C2 在发送前清空 `message`，流敏感分析应怎样处理？

进入下一单元检查点

- ☐ [] 能分别指出数据边和控制边。
- ☐ [] 能解释共享状态为何需要别名分析。
- ☐ [] 能说出传统单函数分析的漏报位置。

第三单元：第3.1—3.3节——从字节码到可分析语义



HScope-图2-系统工作流

图2看图顺序 三段即可：Bytecode Frontend 把包变成 IR；Semantic Analysis Engine 边解释边建立调用与 ICC 关系；Privacy Detection 在这些关系上跑污点规则并生成报告。

3.1 字节码前端与 IR 译读

HScope 输入 OpenHarmony 应用包，使用 ArkCompiler 的 `ark_disasm` 解码 Ark 字节码。前端解析指令并做结构化反编译，把跳转恢复为条件分支等结构，再转换成轻量 IR。它还恢复变量名、类属性与词法环境，使闭包中的槽位能重新对应到变量。

3.2 抽象域译读

真实程序对象无限多，静态分析不能枚举每次运行。HScope 使用抽象内存对象表示“某代码位置可能创建的一组对象”，并把函数、类、对象、闭包和属性的可能值记录在抽象状态中。值集合保守合并：宁可包含多种可能，也不能随意漏掉一种。

3.3 抽象语义译读

对赋值、属性读写、对象创建、条件和函数调用，论文定义状态转移规则。执行到调用点时，系统读取调用变量当前可能指向的函数对象，动态补充调用图，而不是要求开始前已有完整调用图。返回时再把被调函数结果合并回调用者状态。

三个新术语

- ☐ IR (Intermediate Representation, 中间表示)：统一的赋值、属性读写和调用等分析指令。
- ☐ Abstract Object (抽象对象)：代表一组可能运行时对象的有限符号。
- ☐ On-the-fly Call Graph (按需调用图)：分析到调用点时根据当前状态解析目标并加边。

批注：抽象解释不是模拟器

它不追求还原某个具体 OAID 或某次点击，而是计算所有允许状态的安全近似。代价是可能把不可能同时出现的值合并，形成误报。

最少记忆清单

- ☐ 字节码先变成 IR，IR 再被抽象解释。
- ☐ 抽象对象用有限状态覆盖许多真实对象。

- ☐ 调用图在分析过程中逐步形成。

主动回忆题

1. 为什么恢复词法环境对闭包很重要？
2. 抽象值合并为什么带来误报却有助于避免漏报？
3. 固定调用图与按需调用图有什么差别？

进入下一单元检查点

- ☐ [] 能按“包—反汇编—IR—抽象状态—调用图”复述流程。
- ☐ [] 能解释一个抽象对象代表什么。
- ☐ [] 首次阅读可以略过公式，但能说出每类公式在更新什么状态。

第四单元：第3.4—3.7节——上下文、框架与污点传播

3.4 上下文敏感引擎译读

同一函数从不同调用点进入时，参数来源可能完全不同。HScope 使用有限调用点上下文区分这些执行，避免把无关数据流混在一起。引擎还维护摘要以减少重复分析，并在状态变化时继续迭代直到稳定。

3.5 污点分析译读

系统把 Source API 返回值标上数据类别，例如 OAID、位置或联系人。标签沿赋值、属性、参数、返回值与容器传播。若标签到达网络、日志、短信等 Sink，就产生候选路径。对 Map/Record 等复合结构，论文使用汇总近似，这有利于覆盖但可能污染兄弟字段。

3.6 隐私泄漏检测译读

HScope 使用可配置 JSON 策略定义 Source、Sink 和污点类别，并包含 OpenHarmony 框架模型：Ability 生命周期、Router 导航、事件回调以及共享 UI 状态。路径报告不仅说“命中”，还应保留调用和传播位置，供人工复核。

3.7 实现译读

原型以 Python 为主，并配合 tree-sitter 语法和 Ark 工具链。论文版本后来集成进 Lian，可通过指定 abc 语言启用。实现规模说明系统远不只是几条正则表达式。

三个新术语

- ☐ Context-sensitive（上下文敏感）：区分函数从不同调用位置进入的情况。
- ☐ Flow-sensitive（流敏感）：保留语句先后顺序对状态的影响。
- ☐ Taint Summary（污点摘要）：复用函数已经计算过的数据流效果。

最少记忆清单

- ☐ 污点标签只有建立在正确调用与 ICC 关系上才有意义。
- ☐ Source/Sink 策略和框架模型都是知识盲区来源。
- ☐ 上下文与流敏感提高精度，也增加时间和内存。

主动回忆题

1. 为什么同一工具函数需要区分两个调用点？
2. 复合对象汇总近似会怎样产生误报？
3. 若策略漏掉一个新隐私 API，会造成误报还是漏报？

进入下一单元检查点

- ☐ [] 能逐边解释图1的污点传播。
- ☐ [] 能说出 HScope 的三个框架建模对象。
- ☐ [] 能区分通用程序语义与平台专用语义。

第五单元：第4节——实验怎样证明有效

数据与开销译读

作者构建 50 个用例、覆盖 11 类模式的 OH-Bench 微基准，并收集 300 个开源 OpenHarmony App。由于官方应用市场包存在加密和反嗅探限制，真实数据集并非直接抓取全部闭源商店 App。实验机为 i7-14700、32 GB RAM、Ubuntu 24.04；平均每个 App 分析 206 秒，平均内存 1,523 MB。

微基准结果译读

HScope 在 OH-Bench 上 precision 与 recall 均为 87.5%，F1 为 0.88。基线 ArkAnalyzer precision 为 81.8%、recall 为 22.5%、F1 为 0.35。主要提升来自对间接调用、共享状态与框架通信的处理。

真实 App 结果译读

HScope 报告 48 条候选泄漏路径，经人工检查确认 39 条可行，涉及 27 个 App。这里“确认”是研究者根据代码路径判断，不应把 39/48 直接推广为所有环境下的通用准确率。

消融与对比译读

论文分别移除调用上下文和框架建模，观察召回与误报变化。结果说明间接调用解析和框架专用别名关系是关键贡献；仅有通用污点传播不足以恢复真实 OpenHarmony 路径。

三个新术语

- ☐ Micro-benchmark（微基准）：为某种语言或框架特性专门设计的小测试用例。
- ☐ F1-score（F1值）：precision 与 recall 的调和平均。
- ☐ Candidate Path（候选路径）：工具报告、仍需人工判断可行性和风险含义的路径。

最少记忆清单

- ☐ OH-Bench 与 300 个真实 App 回答不同问题。
- ☐ 48 是工具候选，39 是人工确认路径。
- ☐ 平均开销适合离线分析，不表示手机端实时执行。

主动回忆题

1. 为什么微基准高分不能替代真实 App 测试？
2. ArkAnalyzer recall 低说明可能漏掉哪类关系？
3. 39 条确认路径是否等于 39 个法律违规？

进入下一单元检查点

- ☐ [] 能解释 50、300、48、39、27 五个数字。
- ☐ [] 能说出实验机器和平均开销。
- ☐ [] 能解释消融实验怎样支持框架建模贡献。

第六单元：第5—7节——相关工作、局限与结论

相关工作译读

论文把自己放在三条研究线上：JavaScript/动态语言静态分析、Android 与移动隐私污点分析、OpenHarmony 程序分析。HScope 的差异是直接面向 Ark 字节码，并同时处理无类型对象、间接回调和 OpenHarmony ICC。

局限译读

反射、字符串动态导入和运行时生成属性仍可能让目标不可解析。Map/Record 汇总近似可能产生误报。有限上下文深度无法完全覆盖非常深或高度多态的回调。Source/Sink 策略和框架模型也需要持续维护。

结论译读

HScope 说明 OpenHarmony 隐私分析不能只复制 Android 工具。只有恢复字节码语义、调用关系和组件数据关系，污点分析才能给出更完整的路径。真实 App 测量显示该方法具有实际价值，但路径仍需人工和动态证据进一步确认。

三个新术语

- ☐ Reflection（反射）：运行时根据字符串等信息决定类型或方法。
- ☐ Soundness（健全性目标）：尽量不漏掉真实行为的分析追求；现实工具常需折中。
- ☐ False Positive（假阳性）：工具报告但实际不可行或不构成目标风险的候选。

最少记忆清单

- ☐ HScope 的优势与盲区都来自静态近似。
- ☐ 框架升级会要求模型和策略持续更新。
- ☐ 最可靠结论来自静态路径、动态事件和人工复核的组合。

主动回忆题

1. 反射为什么会同时影响调用图和污点传播？
2. 怎样用 Appium/Hook 验证 HScope 的候选路径？
3. HScope 可怎样为 DIULENS 的 GUI 探索提供目标？

完成检查点

- ☐ [] 能用三分钟讲清“字节码—IR—抽象解释—调用图—ICC—污点”。
- ☐ [] 能沿图1讲清每条数据边和控制边。
- ☐ [] 能解释实验数字的样本与分母。
- ☐ [] 能说出至少三项局限。
- ☐ [] 能明确静态路径能证明什么、不能证明什么。

课堂用一句话

HScope 的关键不是把 Source 和 Sink 列出来，而是先恢复 OpenHarmony 无类型字节码中的对象、回调和组件关系，再让污点标签沿这些可验证关系传播。